DALHOUSIE UNIVERSITY

Preprocessing and Visualizing Data

Ordered observations, graphical integrity, and amounts

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Feature engineering and visualization

Feature engineering belongs to data preparation. Visualization belongs to exploratory data analysis. The two feed each other, since a plot can expose a preparation problem and a better-defined variable makes a plot more informative.

Setup for every example

Every later code block assumes this setup has run. R uses `ggplot2` and `tidyr`. Python uses NumPy, pandas and Matplotlib. Install any missing package before running the examples. Plotting examples write named PDF files into your working directory. All small inputs are defined in the code. The Datasaurus example reads a pinned public data file and needs an internet connection.

R

```r
library(ggplot2)
blue <- "#1F5C8B"
```

Python

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import (
    LinearSegmentedColormap)

blue = "#1F5C8B"
```

2 Recording order in lag columns

What it is for A *derived variable* is a new column that expresses a relationship that matters to the question. Examples are an amount per person, a previous measurement, or a change from one interval to the next. This section builds the previous measurement.

2.1 What a row does and does not tell you

A row identifies one observation, and its columns record variables. Neither fact makes the rows independent.

- Readings a minute apart at the same sensor may resemble each other.
- Repeated observations of one person share a subject identity.
- Nearby places may share conditions. A location code alone does not define a neighbourhood.

State what each row represents, and keep the time, subject and location information. Shuffling the rows of a time series can hide the very pattern you want to examine.

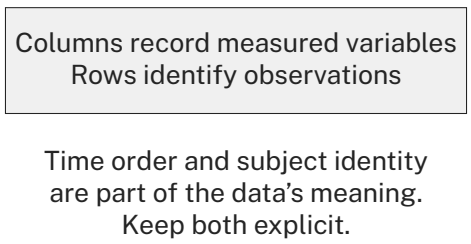


Figure 1. A table structure does not establish independence between rows.

There are two responses to ordered or related observations, and they can be used together. One is to choose an analysis that accounts for order and dependence. The other is to record the relevant

context explicitly as columns, for example by placing an earlier reading next to the current one. This class practises the second. Adding lag columns does not remove serial dependence, and it does not justify a method that assumes independent observations.

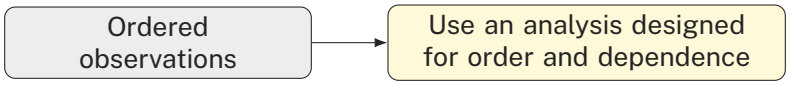


Figure 2. One route is to choose an analysis that represents order and dependence.

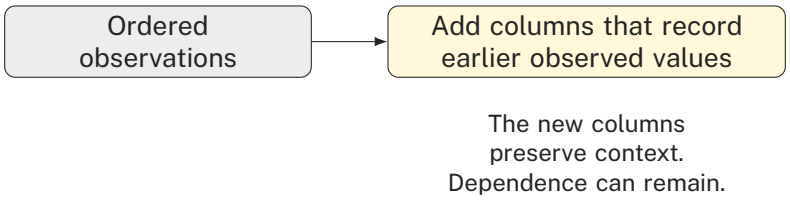


Figure 3. Another route records observed history explicitly. We practise this representation before studying models.

2.2 Time delay embedding

Definition A *lag* is an earlier value in an ordered sequence. If Y_t is the value at time t, then Y_{t-1} is the preceding value and Y_{t-2} the value two intervals earlier. A table holding the current value and several lags is a *time delay embedding*. A complete sequence of n values with k columns, counting the current value, gives $n - k + 1$ complete rows when $1 \leq k \leq n$.

Worked example One subject is observed at seven equally spaced times, with values $4, 5, 6, 7, 8, 9, 10$ in chronological order. With the current value and three lags, $k = 4$. The first three times lack enough history, so the first complete row is at time 4, and the table has $7 - 4 + 1 = 4$ rows.

Table 1. The same chronological lag rows produced by both examples.

Time t	Y_t	Y_{t-1}	Y_{t-2}	Y_{t-3}
4	7	6	5	4
5	8	7	6	5
6	9	8	7	6
7	10	9	8	7

Before creating lags

Sort by time within each subject and decide what one interval means. A row shift is a fixed-duration lag only for equally spaced observations. Never let the final reading of one subject become the first lag of the next. A geographic identifier is not a time order and does not define a spatial neighbourhood.

R

```r
# One subject, equally spaced times, already
# in chronological order.
y <- c(4, 5, 6, 7, 8, 9, 10)
lags <- embed(y, dimension=4)
colnames(lags) <- c("Y", "lag1", "lag2", "lag3")
print(lags)
# Rows 7,6,5,4; 8,7,6,5; 9,8,7,6; 10,9,8,7.
```

Python

```python
# One subject, equally spaced times, already
# in chronological order.
y = pd.Series([4, 5, 6, 7, 8, 9, 10],
              dtype=float)
lags = pd.DataFrame({
    "Y": y, "lag1": y.shift(1),
    "lag2": y.shift(2), "lag3": y.shift(3),
}).dropna()
print(lags.to_numpy())
# Rows [7,6,5,4], [8,7,6,5], [9,8,7,6],
# [10,9,8,7].
```

Differences to know

- R's `embed` builds the whole matrix in one call and returns only the complete rows. The input vector holds doubles, so the matrix holds doubles.
- pandas' `shift` keeps every row and leaves a missing entry where no earlier value exists. `dropna` then removes the three incomplete rows. A real sequence with internal missing values needs its own documented rule.

3 Relative change

What it is for Relative change compares successive amounts on the scale of the starting amount. It answers "how large was the increase relative to where it began", while a plain difference answers "how much was added".

Formula For an amount on a meaningful ratio scale with a nonzero previous value, the fractional relative change is

$$r_t = \frac{Y_t - Y_{t-1}}{Y_{t-1}}.$$

The denominator is the previous amount. Multiply by 100 to report a percentage.

$$6 \longrightarrow 9$$
$$(9 - 6)/6 = 0.5 = 50\%$$

Figure 4. The denominator is the previous amount, not the current amount.

When the formula is invalid Check these before calculating.

- A zero previous value makes the fraction undefined.
- A missing value makes the pair incomplete. Either case needs an explicit handling rule.
- Negative values need a domain-specific interpretation.
- A scale without a meaningful zero, such as temperature in degrees Celsius, does not support a percentage-growth reading.

Worked example A change from 6 to 9 gives $r_t = (9 - 6)/6 = 0.5$, a 50% increase. The change in the original unit is 3. Now take $Y_t = 4(1.5)^t$ for $t = 0, \ldots, 5$, that is $4, 6, 9, 13.5, 20.25, 30.375$. The levels rise, but every successive relative change is 0.5. In the same way, the first differences of an exactly linear sequence are constant. Neither fact means that differencing removes trends, seasonal patterns or dependence from real data.

R

```r
y <- 4 * 1.5^(0:5)
stopifnot(all(!is.na(y)), all(y > 0))
change <- diff(y) / head(y, -1)
print(change)  # Five values equal to 0.5.
print(100 * change)  # 50 percent each interval.
```

Python

```python
y = pd.Series(4 * 1.5 ** np.arange(6))
assert y.notna().all() and (y > 0).all()
change = y.pct_change(fill_method=None)
print(change.tolist())
# Missing first value, then five values
# equal to 0.5.
print((100 * change).tolist())
# Missing first value, then 50 percent each
# interval.
```

Differences to know

- R's `diff` returns five differences from six inputs. Dividing by `head(y, -1)` pairs each difference with its previous value.
- pandas' `pct_change` returns fractional changes despite its name. It keeps all six time positions, with the first change missing because no previous value exists. With `fill_method=None` it does not fill missing observations before calculating.

4 Summaries and plots check each other

What it is for A mean, a standard deviation and a correlation each describe one feature of a dataset. A plot shows what those numbers leave out. Use both.

Definitions The mean describes location and the standard deviation describes spread. Pearson's correlation r, between -1 and 1, describes the direction and strength of a *linear* association. None of them determines the shape of a scatter plot.

4.1 Anscombe's quartet

Worked example Anscombe's four datasets each contain eleven original coordinate pairs.[1] Their selected summaries agree, but their plots do not.

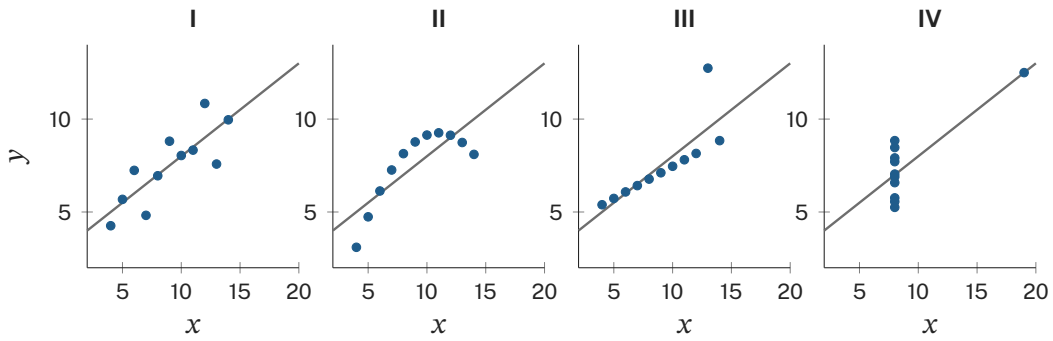


Figure 5. The original quartet on common axes. The grey line is $y = 3 + 0.5x$, a rounded common fitted line shown for comparison. The task is to inspect the points, not to fit a model.

Table 2. The selected summaries agree at this displayed precision, not at every precision.

Summary	Rounded value in each set
Mean x; sample SD x	9.00; 3.32
Mean y; sample SD y	7.50; 2.03
Pearson correlation r	0.82
Approximate fitted line	$y = 3 + 0.5x$

- I shows points scattered around a roughly straight pattern.
- II shows clear curvature.
- III lies close to a straight pattern, with one unusual point above it.
- IV has ten points at the same x. The eleventh has a much larger x and strongly influences the fitted line.

The means, sample standard deviations and correlation (0.82) agree after rounding to two decimals. At three decimals, the fourth correlation rounds to 0.817 and the others to 0.816. Other summaries, such as medians and ranges, need not agree at all.

Code R supplies the original values in `datasets::anscombe`; `lapply` extracts each x, y pair into a list of four tables. Python defines all 44 coordinate pairs in a dictionary with one entry per panel. The plotting loop visits each dataset in turn and draws the same rounded line.

R

```r
# The 44 original coordinates ship with R.
data(anscombe, package="datasets")
quartet <- lapply(1:4, function(i) {
  data.frame(x=anscombe[[i]], y=anscombe[[i+4]])
})
names(quartet) <- c("I", "II", "III", "IV")
```

Python

```python
x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
quartet = {
    "I": (x, [8.04,6.95,7.58,8.81,8.33,9.96,
              7.24,4.26,10.84,4.82,5.68]),
    "II": (x, [9.14,8.14,8.74,8.77,9.26,8.10,
               6.13,3.10,9.13,7.26,4.74]),
    "III": (x, [7.46,6.77,12.74,7.11,7.81,8.84,
                6.08,5.39,8.15,6.42,5.73]),
    "IV": ([8,8,8,8,8,8,8,19,8,8,8],
           [6.58,5.76,7.71,8.84,8.47,7.04,
            5.25,12.50,5.56,7.91,6.89]),
}
```

R

```r
pdf("class-05-anscombe-r.pdf", width=12,
    height=3)
par(mfrow=c(1, 4), mar=c(4, 4, 2, 1))
for (name in names(quartet)) {
  d <- quartet[[name]]
  print(c(mean(d$x), mean(d$y), sd(d$x),
          sd(d$y), cor(d$x, d$y)))
  plot(d$x, d$y, xlim=c(2, 20), ylim=c(2, 14),
       xlab="x", ylab="y", main=name, pch=16,
       col=blue, xaxs="i", yaxs="i")
  abline(a=3, b=.5, col="grey")
}
dev.off()
```

Python

```python
fig, axes = plt.subplots(
    1, 4, figsize=(12, 3),
    sharex=True, sharey=True)
for ax, (name, (x, y)) in zip(
        axes, quartet.items()):
    values = pd.DataFrame({"x": x, "y": y})
    print(name, values.mean().round(2).tolist(),
          values.std(ddof=1).round(2).tolist(),
          round(values.corr().iloc[0, 1], 2))
    ax.scatter(x, y, color=blue)
    ax.plot([2, 20], [4, 13], color="grey")
    ax.set(title=name, xlim=(2, 20),
           ylim=(2, 14), xlabel="x")
axes[0].set_ylabel("y")
fig.tight_layout()
fig.savefig("class-05-anscombe-python.pdf")
plt.close(fig)
```

4.2 The Datasaurus Dozen

Worked example Alberto Cairo created the original dinosaur-shaped dataset. Matejka and Fitzmaurice produced twelve other shapes with nearly the same summaries, giving thirteen sets of 142 points each.[5] The figure plots all $13 \times 142 = 1\,846$ published coordinates with the same axis limits and aspect ratio in every panel.

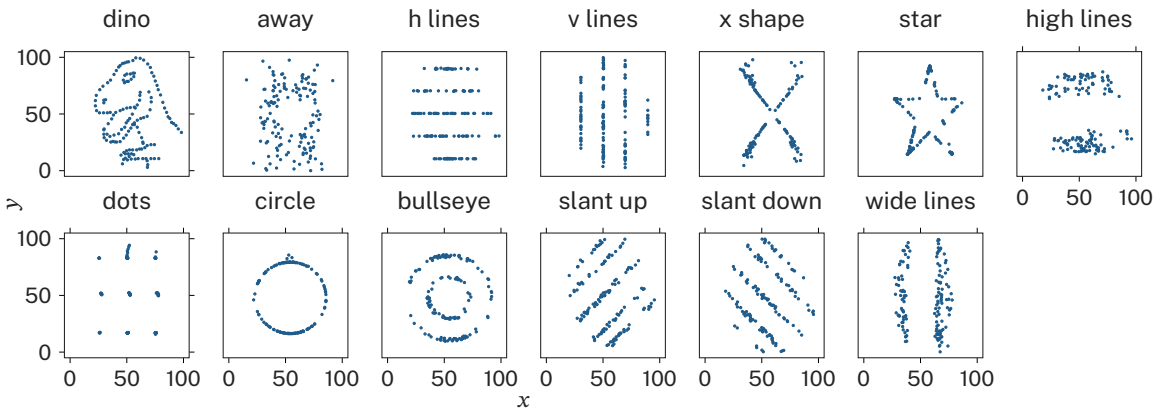


Figure 6. The authentic thirteen datasets on one shared pair of axes. Each panel has 142 points.

Table 3. Ranges calculated across the thirteen sets using sample standard deviations.

Summary	Range across 13 sets
Mean x	54.26 to 54.27
Mean y	47.83 to 47.84
Sample SD x	16.76 to 16.77
Sample SD y	26.93 to 26.94
Pearson r	-0.07 to -0.06

The summaries are nearly equal but not identical at two decimals, so the table reports their ranges. The correlations are small and negative, yet the circles, crosses and stripes are obvious. A weak linear association does not establish independence or rule out a nonlinear pattern.

How to use both Ask what each summary measures and what it ignores. Inspect the plot for shape, gaps, subgroups and unusual observations. Return to the data dictionary before treating an unusual point as an error. Report a pattern as evidence about these data, with its limits.

Code Both languages read the same pinned revision of the public TSV file in the datasauRus package [4], so the labels, row counts and coordinates match. Python needs no R installation.

R

```r
revision <-
  "0496ac15208e9ee4a58ea81a5de46912f095aa15"
url <- paste0(
  "https://raw.githubusercontent.com/",
  "jumpingrivers/datasauRus/", revision,
  "/inst/extdata/DatasaurusDozen-Long.tsv")
dino <- read.delim(url)
stopifnot(nrow(dino) == 1846)
print(aggregate(cbind(x, y) ~ dataset, dino,
                mean))
print(aggregate(cbind(x, y) ~ dataset, dino,
                sd))
correlations <- by(
  dino, dino$dataset,
  function(part) cor(part$x, part$y))
print(correlations)
# Sample SD; values are close, not identical
# after ordinary rounding.
```

Python

```python
revision = (
    "0496ac15208e9ee4a58ea81a5de46912f095aa15")
url = ("https://raw.githubusercontent.com/"
       "jumpingrivers/datasauRus/" + revision
       + "/inst/extdata/"
       "DatasaurusDozen-Long.tsv")
dino = pd.read_csv(url, sep="\t")
assert len(dino) == 1846
summaries = (dino.groupby("dataset")[["x", "y"]]
             .agg(["mean", "std"]))
print(summaries.round(3))
for name, part in dino.groupby("dataset"):
    print(name, part["x"].corr(part["y"]))
# Sample SD; values are close, not identical
# after ordinary rounding.
```

R

```r
p <- ggplot(dino, aes(x, y)) +
  geom_point(size=.6, colour=blue) +
  facet_wrap(~dataset, ncol=5) +
  coord_fixed(xlim=c(0, 100), ylim=c(0, 100),
              expand=FALSE) +
  theme_minimal()
ggsave("class-05-datasaurus-r.pdf", p, width=12,
       height=7)
```

Python

```python
fig, axes = plt.subplots(
    3, 5, figsize=(12, 7),
    sharex=True, sharey=True)
for ax, (name, part) in zip(
        axes.flat,
        dino.groupby("dataset", sort=True)):
    ax.scatter(part["x"], part["y"], s=5,
               color=blue)
    ax.set(title=name, xlim=(0, 100),
           ylim=(0, 100), aspect="equal")
for ax in axes.flat[13:]:
    ax.set_visible(False)
fig.tight_layout()
fig.savefig("class-05-datasaurus-python.pdf")
plt.close(fig)
```

5 Choosing a chart from a question

What it is for The question decides what the reader compares. The variable type, units and meaningful order then constrain how it is drawn.

Rule State the question in one sentence first. Then choose the variables and the chart family that answer it. The families follow Wilke's directory of visualizations [7]. The same numeric column can be an amount compared across groups, an observation in a distribution, a part of a total, or one coordinate in an association.

Table 4. Five useful question families. This class develops amounts.

Question	Starting points
How much in each category?	Bars, dots, grouped or stacked bars, heatmaps.
How are values distributed?	Histograms, densities, box plots, violins, ECDFs, Q-Q plots.
How is a whole divided?	Bars, pies, stacked bars; stacked areas over time.
How do two variables relate?	Scatter plots; a line when order has meaning.
How do several variables relate?	Scatter-plot matrices, correlograms, parallel coordinates.

5.1 Defaults are choices made by software

Rule A call that asks for “a histogram” also chooses break locations, the treatment of missing values and the axis limits. Record library versions and make consequential choices explicit.

Table 5. Default bin rules under the standard configuration. Configuration and library versions can change them.

Interface	Default bin choice
R `hist`	Sturges’ rule, then convenient break locations.
ggplot2 `geom_histogram`	30 bins if no bin choice is supplied.
Matplotlib `hist`	10 bins under the standard configuration.
seaborn `histplot`	NumPy’s `auto` bin rule.

Worked example To compare languages fairly, use the same values, the same edges and the same missing-value policy. The input is the eight numbers $2, 4, 4, 5, 7, 9, 12, 41$, plus one missing entry. Both programs drop the missing entry and say so in the title. The edges are $0, 10, 20, 30, 40, 50$, with intervals closed on the left and the final right endpoint included. Six values fall in $[0, 10)$, 12 falls in $[10, 20)$, and 41 falls in $[40, 50]$. No value lies on an interior edge, and both programs return the counts $6, 1, 0, 0, 1$.

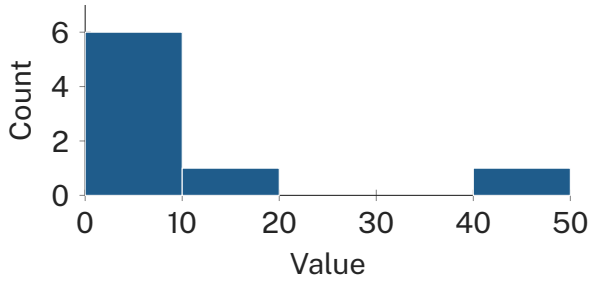


Figure 7. A single fixed dataset and bin specification gives the same counts in both languages.

R

```r
values <- c(2, 4, 4, 5, 7, 9, 12, 41, NA)
observed <- values[!is.na(values)]
edges <- c(0, 10, 20, 30, 40, 50)
pdf("class-05-histogram-r.pdf")
h <- hist(
  observed, breaks=edges, right=FALSE,
  col=blue, border="white",
  xlab="Value", ylab="Count",
  main="Eight observed values; one missing")
print(h$counts)  # 6, 1, 0, 0, 1
dev.off()
```

Python

```python
values = pd.Series([2, 4, 4, 5, 7, 9, 12, 41,
                    np.nan])
observed = values.dropna()
edges = [0, 10, 20, 30, 40, 50]
fig, ax = plt.subplots()
counts, _, _ = ax.hist(observed, bins=edges,
                       color=blue,
                       edgecolor="white")
print(counts.astype(int).tolist())
# 6, 1, 0, 0, 1
ax.set(
    xlabel="Value", ylabel="Count",
    title="Eight observed values; one missing")
fig.savefig("class-05-histogram-python.pdf")
plt.close(fig)
```

Common bins separate a real difference in data from a difference in library defaults. Five bins are not the best choice for every purpose.

6 Graphical integrity

What it is for A correct number can still be drawn in a way that misleads. These rules keep the visual size of a comparison in line with its numeric size.

6.1 Bar lengths need their baseline

Rule The length of a bar is read as the amount. Start bars at zero so that length is proportional to the stated value.

Worked example A teaching redraw compares two rates, the 2012 top income tax rate of 35% and the 39.6% scheduled for 2013. The difference is $39.6 - 35 = 4.6$ percentage points. Relative to the first rate, the increase is

$$\frac{39.6 - 35}{35} \times 100\% \approx 13.1\%,$$

so the second rate is about 1.13 times the first. If the bars are clipped at 34%, their visible lengths are $35 - 34 = 1$ and $39.6 - 34 = 5.6$ percentage points. The second bar then looks 5.6 times as long.

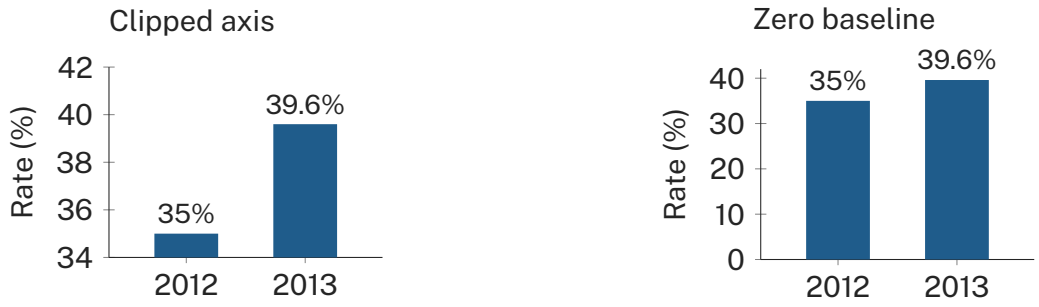


Figure 8. The same two rates. Both show an increase; the clipped baseline exaggerates the apparent magnitude when readers compare bar lengths. This is a teaching redraw, not a reproduced historical broadcast; the two rates come from a widely circulated 2012 US television chart of the top income tax rate.

A dot plot may use a narrower, clearly labelled axis, because position, not length from an implied origin, carries the comparison. That axis still needs context and must not hide relevant values.

6.2 Four integrity checks

Rule These checks adapt Tufte's discussion of graphical integrity.[6] They apply to exploratory plots as well as finished ones. If a plotting choice changes the apparent conclusion, document it and inspect an alternative.

Table 6. Questions to ask before accepting a plot.

Match visual magnitude to numeric magnitude.	For bars, begin at zero so length represents the stated amount.
Label the comparison completely.	Give the variable, unit, population, time period, and source.
Show variation in data, not decoration.	Keep scales and encodings consistent across comparable panels.
Give enough context to interpret the result.	Report exclusions and avoid selecting a range that conceals the pattern.

7 Visual encodings

What it is for An *encoding* maps a value or a category to a visible property such as position, length, area, angle, colour intensity, hue or shape. Choose the encoding for the comparison the reader must make.

7.1 Magnitude and category identity are different tasks

Rule For precise comparisons of numeric magnitude, aligned position and length are easier to judge than area or angle, as Cleveland and McGill's perception experiments showed [3]. Identifying a category is a different task. Hue, shape, line style and text distinguish groups well without implying that one group is larger.

Compare numeric values

Aligned position and length usually support more precise comparisons than area, angle, or intensity.

Identify groups

Use hue, shape, line style, or text to distinguish groups. Categories do not need a magnitude ranking.

Figure 9. Choose the encoding for the comparison the reader needs to make.

Table 7. Magnitude comparisons and categorical identification need different cues.

Task	Useful encodings
Precise magnitude	Aligned position or length with a labelled scale.
Approximate magnitude	Area, angle, or intensity when they suit the question.
Category identity	Hue, shape, line style, and direct labels.

Worked example The next figure shows the values 2, 4 and 8. With aligned dots and bars, the ratios are easy to read. The circles encode the same numbers by area, so each radius grows with the square root of the value. Doubling a radius would quadruple the area. A pie chart also uses angle and area. It can suit a simple part-to-whole question, but aligned bars make close comparisons easier.

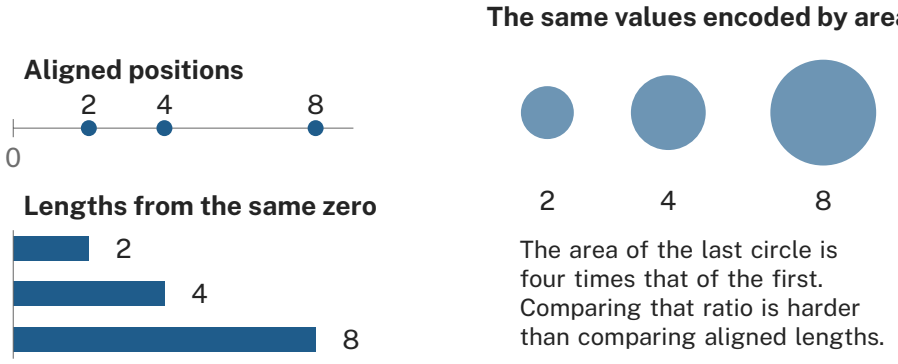


Figure 10. The same three values shown through position, length, and area. Aligned positions make the ratio between the values easier to compare.

7.2 Use more than colour to convey meaning

Rule Colour must not be the only carrier of meaning, as the W3C guidance on colour requires. Add direct labels, different shapes or distinct line styles, so groups stay identifiable when hues are hard to tell apart or the chart is printed in grayscale.

Worked example Group A has circles and a solid line, group B has squares and a dashed line, and labels at the line ends name both. The groups remain distinguishable in monochrome.

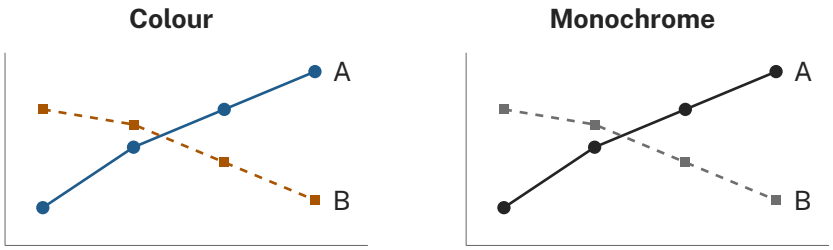


Figure 11. The labelled groups remain distinguishable in colour and monochrome. This comparison is not a physiological simulation of any particular colour-vision condition.

A monochrome check is evidence about redundant encoding, not proof of accessibility for every reader. A palette alone does not make a chart accessible; label size, contrast, descriptions and the final display size also matter. Inspect the plot at the size readers will see it, and write a caption that states the comparison.

8 Bar and dot plots of amounts

What it is for Bars or dots compare one amount across categories. The sequence below, from rotated labels to heatmaps, follows Wilke's treatment of amounts [7]. This section applies the integrity and encoding rules of sections 6 and 7 to one real table.

8.1 The data

The table records regular-season wins for the five Atlantic Division teams over four seasons. Each team-season pair has one already summarized amount. The first question is which team won most in 2018–19.

Table 8. Atlantic Division regular-season wins. Totals sum the four listed seasons, not a single season.

Team	2015–16	2016–17	2017–18	2018–19	Total
Boston Celtics	48	53	55	49	205
Brooklyn Nets	21	20	28	42	111
New York Knicks	32	31	29	17	109
Philadelphia 76ers	10	28	52	51	141
Toronto Raptors	56	51	59	58	224

The twenty values were checked against NBA season records. The NBA's draft-order releases report the final records for 2015–16, 2016–17, and 2018–19; its 2018 Eastern Conference review reports the 2017–18 records. Read the wins in each team's record, not its draft position or the team receiving its pick.

The setup below defines the same twenty values for every bar, dot, grouped and stacked example. It also sorts the rows from fewest to most 2018–19 wins.

R

```r
nba <- data.frame(
  Team=c("Boston Celtics", "Brooklyn Nets",
         "New York Knicks",
         "Philadelphia 76ers", "Toronto Raptors"),
  `2015-16`=c(48, 21, 32, 10, 56),
  `2016-17`=c(53, 20, 31, 28, 51),
  `2017-18`=c(55, 28, 29, 52, 59),
  `2018-19`=c(49, 42, 17, 51, 58),
  check.names=FALSE
)
ordered <- nba[order(nba[["2018-19"]]), ]
ordered$Team <- factor(ordered$Team,
                       levels=ordered$Team)
# Small values at the bottom; largest at the
# top of a horizontal plot.
```

Python

```python
nba = pd.DataFrame({
    "Team": ["Boston Celtics", "Brooklyn Nets",
             "New York Knicks",
             "Philadelphia 76ers",
             "Toronto Raptors"],
    "2015-16": [48, 21, 32, 10, 56],
    "2016-17": [53, 20, 31, 28, 51],
    "2017-18": [55, 28, 29, 52, 59],
    "2018-19": [49, 42, 17, 51, 58],
})
ordered = nba.sort_values("2018-19")
# Small values at the bottom; largest at the
# top of a horizontal plot.
```

8.2 Data, mappings, marks and labels

Definitions Every chart here is built from four parts, whatever the language.

- *Data* supply the values, here five teams' 2018–19 wins.
- A *mapping* assigns team to a categorical position and wins to a numeric position.
- A *mark*, a bar or a dot, represents each amount.
- *Labels and scales* state the season and unit, keep names readable, and fix a deliberate range.

In R, `ggplot` supplies data and mappings, `geom_col` adds a bar layer for already summarized values, and labels, scales and a theme complete the plot. In Python, a Matplotlib figure contains axes, `ax.barh` adds bars to the axes, and axis methods set the labels, limits and title. Python also has layered interfaces, such as seaborn's objects interface. The syntax differs; the visual choices are the same.

A function that aggregates observations answers a different question from one that draws a supplied value. Turning off seaborn's error bars does not turn off its aggregation. Here the five supplied values are drawn directly. If a table has several rows per team, decide how to summarize them first.

8.3 Order and orientation

Rule Alphabetical order helps a reader find a named category. Ordering nominal categories by amount helps a ranking question. If the categories have an inherent order, such as an ordinal rating or a sequence of dates, keep that order instead of sorting by size. Orientation is a separate problem. Long names do not fit under narrow vertical bars, and rotating them makes every name harder to read.

Worked example The first two charts rotate the names, the first in alphabetical order and the second in order of wins.

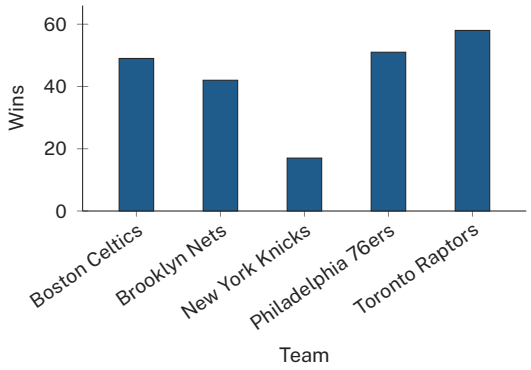


Figure 12. Alphabetical order supports lookup. The long names fit only because they are rotated.

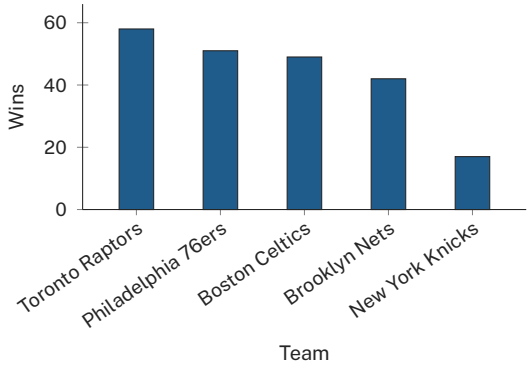


Figure 13. Ordering by wins clarifies the ranking, but the rotated names are still read at an angle.

Turning the bars horizontal gives the names room. The final chart puts the largest amount at the top and keeps a zero baseline. Toronto has 58 wins, Philadelphia 51, Boston 49, Brooklyn 42 and New York 17.

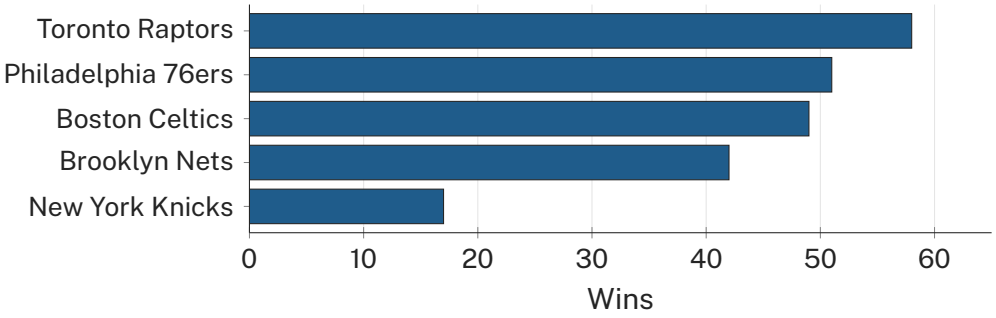


Figure 14. Readable names and a deliberate ranking on the same zero-based scale.

R

```r
p <- ggplot(ordered, aes(x=`2018-19`, y=Team)) +
  geom_col(fill=blue) + xlim(0, 65) +
  labs(x="Wins", y=NULL,
       title="Atlantic Division, 2018-19") +
  theme_minimal()
ggsave("class-05-bars-r.pdf", p, width=7,
       height=4)
```

Python

```python
fig, ax = plt.subplots()
ax.barh(ordered["Team"], ordered["2018-19"],
        color=blue)
ax.set(xlabel="Wins",
       title="Atlantic Division, 2018-19",
       xlim=(0, 65))
fig.tight_layout()
fig.savefig("class-05-bars-python.pdf")
plt.close(fig)
```

Differences to know

- In R, `ordered` stores the teams as factor levels from fewest to most wins, and ggplot2 draws the last level at the top.
- Matplotlib's categorical axis draws the first supplied name at the bottom. The ascending row order therefore also puts Toronto at the top.
- Setting the order explicitly avoids an accidental reversal or an alphabetical default in either language.

8.4 A dot represents the same amount

Rule A dot plot uses position on a common scale. It helps when bar lengths add little, or when a clearly labelled narrower axis is needed to separate close values.

Worked example This chart keeps the bar chart's 0–65 range so that only the mark changes. Each dot lies exactly on its team's row, with no jitter or aggregation.

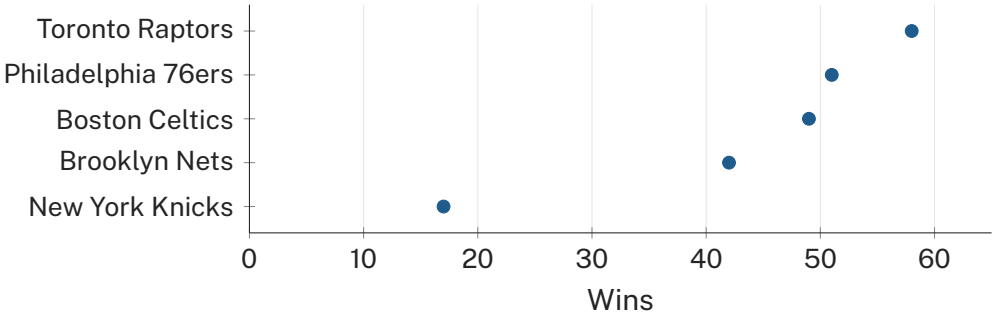


Figure 15. The same five 2018–19 amounts represented by positions.

R

```r
p <- ggplot(ordered, aes(x=`2018-19`, y=Team)) +
  geom_point(colour=blue, size=3) +
  xlim(0, 65) +
  labs(x="Wins", y=NULL,
       title="Atlantic Division, 2018-19") +
  theme_minimal()
ggsave("class-05-dots-r.pdf", p, width=7,
       height=4)
```

Python

```python
fig, ax = plt.subplots()
ax.scatter(ordered["2018-19"], ordered["Team"],
           color=blue)
ax.set(xlabel="Wins",
       title="Atlantic Division, 2018-19",
       xlim=(0, 65))
fig.tight_layout()
fig.savefig("class-05-dots-python.pdf")
plt.close(fig)
```

9 Several amounts per category

What it is for With four seasons per team there are two questions. How did the seasons compare for each team, and how do the four-season totals compare? Grouped and stacked bars answer one each.

Rule Grouped bars place each season on a common zero, so individual seasons are easy to compare. Stacked bars place the seasons end to end, so the total is easy to compare and each season's contribution is visible. Interior stacked segments have no shared baseline, which makes close comparisons

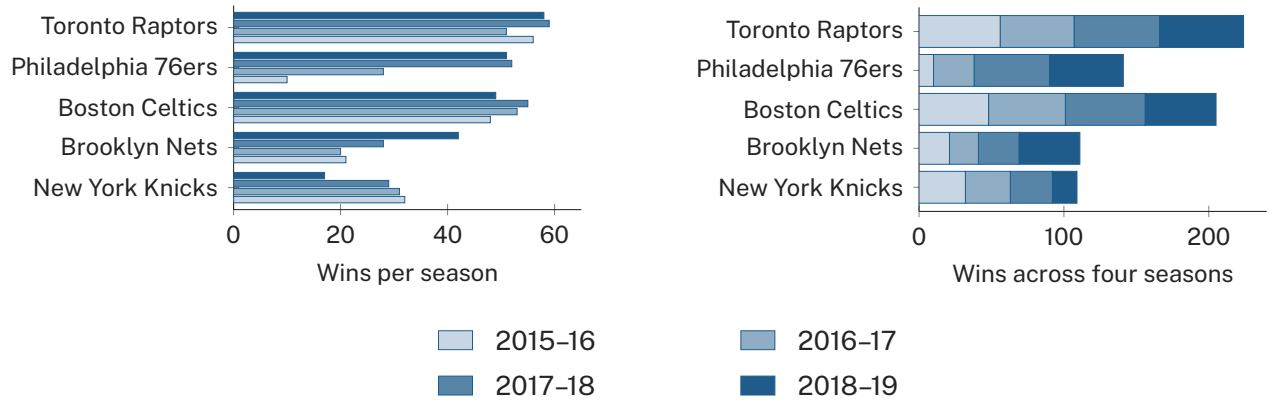


Figure 16. The same twenty values in grouped and stacked form. Seasons run 2015–16 to 2018–19, bottom to top within groups and left to right within stacks. Later seasons are darker, and position also identifies each season.

between them harder. Stack only amounts that can be added meaningfully, as wins across distinct seasons can.

Table 9. The comparison changes with the arrangement of the bars.

Display	Comparison supported
Grouped	Compare individual seasons from a common zero.
Stacked	Compare totals and their components; interior segments lack a shared baseline.

Worked example The four-season totals are Toronto 224, Boston 205, Philadelphia 141, Brooklyn 111 and New York 109. This ranking differs from the 2018–19 ranking, where Philadelphia (51) was ahead of Boston (49). The two charts answer different questions, so the stacked chart's axis label names the four-season period.

Code The first step reshapes the table. R turns the wide table into one row per team and season, and a `Season` factor fixes the chronological order. Python keeps a wide table with the seasons as columns in the same order. Both shapes hold the same twenty observations, and no missing team-season pair is filled with zero.

R

```r
seasons <- names(nba)[-1]
nba_long <- tidyr::pivot_longer(
  ordered, cols=-Team,
  names_to="Season", values_to="Wins")
nba_long$Season <- factor(nba_long$Season,
                          levels=seasons)
# One hue, darker for later seasons, so the
# seasons also differ in lightness.
colors <- c("#C7D6E2", "#8FAEC5", "#5785A8", blue)
```

Python

```python
seasons = list(nba.columns[1:])
wide = ordered.set_index("Team")[seasons]
# One hue, darker for later seasons, so the
# seasons also differ in lightness.
colors = ["#C7D6E2", "#8FAEC5", "#5785A8", blue]
```

Draw the grouped bars first. The widths and offsets keep the four seasons inside each team's row.

R

```r
p <- ggplot(nba_long,
            aes(x=Wins, y=Team, fill=Season)) +
  geom_col(position=position_dodge(width=.8),
           width=.7) +
  scale_fill_manual(values=colors,
                    name="Season\n(bottom to top)") +
  xlim(0, 65) +
  labs(x="Wins per season", y=NULL) +
  theme_minimal()
ggsave("class-05-grouped-r.pdf", p, width=8,
       height=5)
```

Python

```python
ax = wide.plot.barh(color=colors, width=0.8)
ax.set(xlabel="Wins per season", xlim=(0, 65))
ax.legend(title="Season (bottom to top)",
          ncols=1, loc="upper left",
          bbox_to_anchor=(1, 1))
ax.figure.tight_layout()
ax.figure.savefig("class-05-grouped-python.pdf")
plt.close(ax.figure)
```

Then stack the same seasons.

R

```r
p <- ggplot(nba_long,
            aes(x=Wins, y=Team, fill=Season)) +
  geom_col(
    position=position_stack(reverse=TRUE),
    width=.7) +
  scale_fill_manual(values=colors,
                    name="Season\n(left to right)") +
  xlim(0, 240) +
  labs(x="Wins across four seasons", y=NULL) +
  theme_minimal()
ggsave("class-05-stacked-r.pdf", p, width=8,
       height=5)
```

Python

```python
ax = wide.plot.barh(stacked=True, color=colors,
                    width=0.7)
ax.set(xlabel="Wins across four seasons",
       xlim=(0, 240))
ax.legend(title="Season (left to right)",
          ncols=1, loc="upper left",
          bbox_to_anchor=(1, 1))
ax.figure.tight_layout()
ax.figure.savefig("class-05-stacked-python.pdf")
plt.close(ax.figure)
```

Differences to know

- R works from the long table. For grouped bars it uses `position_dodge`. For stacks it uses `position_stack(reverse=TRUE)`, so the segments run left to right in chronological order.
- pandas works from the wide table. Each season is one column of `wide`, and segments stack in column order. The same call without `stacked=True` produces grouped bars.

10 Heatmaps

What it is for A heatmap shows a whole matrix of amounts at once. It combines two categorical positions, the row and the column, with a colour encoding for the value.

Definition Each cell maps one numeric value to a colour. The row and column labels identify the pair the cell represents, and the colour key states the numeric mapping. Heatmaps are common in RNA analysis, where rows may be genes and columns tissues or samples. The meaning of a colour depends on the measurement and on any transformation applied.

Worked example The matrix has seven genes and seven tissues. The values are deviations from a chosen reference in arbitrary units, not row-standardized scores. All 49 values are printed in the cells and defined in the code.

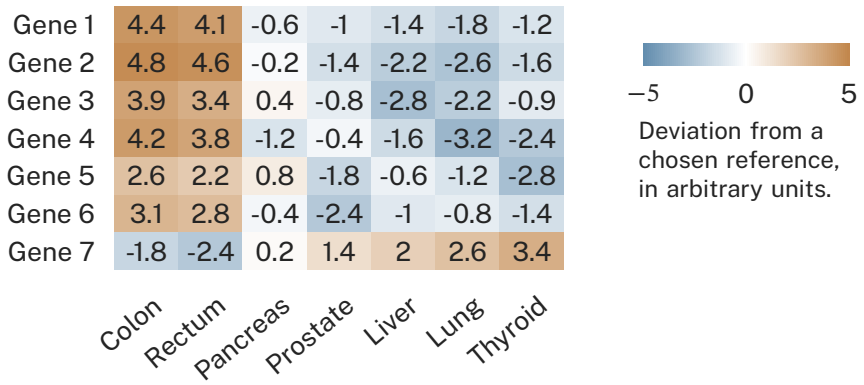


Figure 17. A matrix with an explicit reference. Blue represents negative deviations, white represents zero, and orange represents positive deviations. Layout after the Wikimedia Commons example RNA-seq heatmap [2].

The colour range is fixed at -5 to 5, with white at zero. White therefore means the chosen reference value. It does not mean the overall mean, a row mean, or an absence of expression. If a later analysis standardizes each row, zero then represents that row's mean, but that is a different, stated transformation. Apply and check the standardization formula before making that claim.

Reading the plot means explaining the value in one cell, the zero reference, and one visible pattern. Genes 1 to 6 are above the reference in Colon and Rectum and mostly below it elsewhere, while Gene 7 shows the opposite pattern.

R

```r
# Deviations from a chosen reference, in
# arbitrary units. Rows are genes; columns
# are tissues.
values <- rbind(
  c(4.4,4.1,-.6,-1,-1.4,-1.8,-1.2),
  c(4.8,4.6,-.2,-1.4,-2.2,-2.6,-1.6),
  c(3.9,3.4,.4,-.8,-2.8,-2.2,-.9),
  c(4.2,3.8,-1.2,-.4,-1.6,-3.2,-2.4),
  c(2.6,2.2,.8,-1.8,-.6,-1.2,-2.8),
  c(3.1,2.8,-.4,-2.4,-1,-.8,-1.4),
  c(-1.8,-2.4,.2,1.4,2,2.6,3.4)
)
tissues <- c("Colon", "Rectum", "Pancreas",
             "Prostate", "Liver", "Lung",
             "Thyroid")
```

Python

```python
# Deviations from a chosen reference, in
# arbitrary units. Rows are genes; columns
# are tissues.
values = np.array([
    [4.4,4.1,-.6,-1,-1.4,-1.8,-1.2],
    [4.8,4.6,-.2,-1.4,-2.2,-2.6,-1.6],
    [3.9,3.4,.4,-.8,-2.8,-2.2,-.9],
    [4.2,3.8,-1.2,-.4,-1.6,-3.2,-2.4],
    [2.6,2.2,.8,-1.8,-.6,-1.2,-2.8],
    [3.1,2.8,-.4,-2.4,-1,-.8,-1.4],
    [-1.8,-2.4,.2,1.4,2,2.6,3.4],
])
tissues = ["Colon", "Rectum", "Pancreas",
           "Prostate", "Liver", "Lung",
           "Thyroid"]
```

R

```r
cells <- expand.grid(Gene=1:7, Tissue=1:7)
cells$Value <- as.vector(values)
cells$Gene <- factor(cells$Gene, levels=7:1)
cells$Tissue <- factor(cells$Tissue,
                       labels=tissues)
p <- ggplot(cells,
            aes(Tissue, Gene, fill=Value)) +
  geom_tile() +
  geom_text(aes(label=Value), colour="black",
            size=3) +
  scale_fill_gradient2(
    low="#628DAE", mid="white", high="#C2874D",
    midpoint=0, limits=c(-5, 5),
    name="Deviation\n(arbitrary units)") +
  labs(title="Deviation matrix",
       x=NULL, y="Gene") +
  theme_minimal() +
  theme(axis.text.x=element_text(angle=40,
                                 hjust=1))
ggsave("class-05-heatmap-r.pdf", p, width=8,
       height=5)
```

Python

```python
palette = LinearSegmentedColormap.from_list(
    "deviation", ["#628DAE", "white", "#C2874D"])
fig, ax = plt.subplots()
image = ax.imshow(values, cmap=palette,
                  vmin=-5, vmax=5)
for row in range(7):
    for column in range(7):
        ax.text(column, row,
                f"{values[row, column]:g}",
                ha="center", va="center",
                color="black", fontsize=8)
ax.set_xticks(range(7), tissues, rotation=40,
              ha="right")
ax.set_yticks(
    range(7), [f"Gene {i}" for i in range(1, 8)])
fig.colorbar(
    image, ax=ax,
    label="Deviation from reference "
          "(arbitrary units)")
ax.set_title("Deviation matrix")
fig.tight_layout()
fig.savefig("class-05-heatmap-python.pdf")
plt.close(fig)
```

11 Summary

- A derived variable needs a clear definition and valid inputs.
- Lag columns record history but do not remove dependence. Relative change needs a meaningful, nonzero baseline.
- Plots and summaries describe different aspects of the same data. Use both.
- Make the question, amount, unit, order and grouping visible. Bars start at zero.
- Use aligned position or length for magnitude, and more than colour to identify groups.

References

[1] F. J. Anscombe. “Graphs in statistical analysis”. In: *The American Statistician* 27.1 (1973), pp. 17–21.

[2] E.-S. Choi. *Heat Maps of RNAseqV2*. Wikimedia Commons, file Heatmap_RNAseqV2_1.png. CC BY-SA 4.0. 2015.

[3] W. S. Cleveland and R. McGill. “Graphical perception: theory, experimentation, and application to the development of graphical methods”. In: *Journal of the American Statistical Association* 79.387 (1984), pp. 531–554.

[4] C. Gillespie, S. Locke, R. Davies, and L. D’Agostino McGowan. *datasauRus: Datasets from the Datasaurus Dozen*. Version 0.1.9. CRAN. 2025.

[5] J. Matejka and G. Fitzmaurice. “Same stats, different graphs: generating datasets with varied appearance and identical statistics through simulated annealing”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 2017, pp. 1290–1294.

[6] E. R. Tufte. *The Visual Display of Quantitative Information*. 2nd ed. Graphics Press, 2001.

[7] C. O. Wilke. *Fundamentals of Data Visualization*. O’Reilly Media, 2019.